Cars

By: Sean and Oswal

Our dataset got deleted

Audience

- The class to show how removing different features changed
- how even different types of linear regression can get different results
- how even a simple linear regression can be better than a more complex neural network

- chinese car makers coming into American market (is what the dataset is for if you get models working)

Linear Models

- Linear Regression good for:
  - small data size (ours is about 205 which is pretty small)
  - It should mostly be linear
  - No Regularization
- RidgeCV
  - L2 (Ridge) Regularization
  - Built-in Cross-Validation
  - Helps prevent overfitting
  - Alphas parameter: Regularization strength

Linear Models

- Elastic Net
  - Uses L1 (Lasso) and L2 (Ridge) Regularization
  - Alpha parameter: multiplies the penalty terms
  - L1_ratio parameter: L1 if ratio = 0, L2 if ratio = 1, 0 < ratio < 1 is a combination of L1 and L2
- ARD Regression
  - A Bayesian (Automatic Relevance Determination) Regressor
- SGD Regression
  - Could use L1 and L2 Regularization

Linear Models Results

```
X simple linear regression R-square : 0.8881

X RidgeCV linear regression R-square : 0.8836

X ElasticNet linear regression R-square : 0.888

X ARDRegression linear regression R-square : 0.8831

X SGDRegression linear regression R-square : 0.8835
```

Parameter Checking

- Trying to find if which parameters lead to a better R^2
- removed ones that had zero between
- negative t
- small t (t values smaller than or very close to 1)
- p bigger than 0.5
- p bigger than 0.3
- p bigger than 0.1
- p bigger than 0.05
- p smaller than 0.5
  - to show a point

OLS Regression Results

Dep. Variable:	y	R-squared:	0.888
Model:	OLS	Adj. R-squared:	0.873
Method:	Least Squares	F-statistic:	59.52
Date:	Tue, 18 Mar 2025	Prob (F-statistic):	1.09e-72
Time:	18:23:57	Log-Likelihood:	-1908.0
No. Observations:	205	AIC:	3866.
Df Residuals:	180	BIC:	3949.
Df Model:	24		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	1.328e+04	198.708	66.815	0.000	1.29e+04	1.37e+04
X[0]	-2.2323	342.203	-0.007	0.995	-677.477	673.013
X[1]	-815.7522	246.362	-3.311	0.001	-1301.882	-329.623
X[2]	-176.8307	2030.387	-0.087	0.931	-4183.252	3829.591
X[3]	151.2800	357.555	0.423	0.673	-554.258	856.818
X[4]	328.1144	335.317	0.979	0.329	-333.544	989.773
X[5]	-614.0489	337.522	-1.819	0.071	-1280.058	51.961
X[6]	485.2816	317.662	1.528	0.128	-141.540	1112.103
X[7]	1337.4335	265.948	5.029	0.000	812.657	1862.210
X[8]	865.2419	652.001	1.327	0.186	-421.307	2151.790
X[9]	-689.4269	696.079	-0.990	0.323	-2062.952	684.099
X[10]	1568.0720	548.621	2.858	0.005	485.516	2650.628
X[11]	438.6377	332.227	1.320	0.188	-216.922	1094.198
X[12]	1170.4821	866.365	1.351	0.178	-539.056	2880.021
X[13]	80.8250	237.585	0.340	0.734	-387.985	549.635
X[14]	-1096.3987	744.470	-1.473	0.143	-2565.409	372.612
X[15]	4911.4268	1114.032	4.409	0.000	2713.184	7109.669
X[16]	-30.0973	318.173	-0.095	0.925	-657.926	597.731
X[17]	-895.2081	425.397	-2.104	0.037	-1734.615	-55.801
X[18]	-1114.0488	290.760	-3.832	0.000	-1687.785	-540.312
X[19]	364.3658	1941.324	0.188	0.851	-3466.315	4195.047
X[20]	1185.1284	748.250	1.584	0.115	-291.342	2661.599
X[21]	710.3742	328.079	2.165	0.032	62.999	1357.749
X[22]	-785.3521	1138.263	-0.690	0.491	-3031.408	1460.704
X[23]	727.6964	1056.438	0.689	0.492	-1356.900	2812.293

Omnibus:	41.037	Durbin-Watson:	0.850
Prob(Omnibus):	0.000	Jarque-Bera (JB):	211.590
Skew:	0.602	Prob(JB):	1.13e-46
Kurtosis:	7.829	Cond. No.	40.7

Parameter R^2

These are made with simple linear regression regression models so we want to get a R^2 bigger than 0.8881

None of them gets better than 0.8881 but P bigger than 0.5 and P bigger than 0.3 got fairly close. (we will use those for testing)

P smaller than 0.5 makes sense that it will be quite worse since by excluding the P values smaller than 0.5 we should only really be getting the features that aren't too good

```
X zero between linear regression R-square : 0.8612

X t Negative linear regression R-square : 0.8653

X t Small linear regression R-square : 0.8632

X P bigger than 0.5 linear regression R-square : 0.8858

X P bigger than 0.3 linear regression R-square : 0.8835

X P bigger than 0.1 linear regression R-square : 0.832

X P bigger than 0.05 linear regression R-square : 0.829

X P smaller than 0.5 linear regression R-square : 0.3202
```

Parameter MSE

We see that R^2 closer to 1 get a lower MSE.

We can see that the MSE for P bigger than 0.5 and P bigger than 0.3 give us the best results while P smaller than 0.5 gets the worst results.

```
X zero between gets a MSE of 8814067.1586

X negative t gets a MSE of 8556432.8798

X small t gets a MSE of 8688871.792

X P bigger than 0.5 gets a MSE of 7255746.4066

X P bigger than 0.3 gets a MSE of 7395836.6957

X P bigger than 0.1 gets a MSE of 10667091.5712

X P bigger than 0.05 gets a MSE of 10861488.0532

X P smaller than 0.5 gets a MSE of 43172383.9172
```

Comparisons

Now we will compare several different regression models and how how they compare to each other.

We will use all 3 different sets of features that we believe will have the best results.

- All the features
- Remove features with a P bigger than 0.5
- Remove features with a P bigger than 0.3

(this will be tested later as well)

Simple Linear Regression

- We see that Simple Linear Regression with all the features gets a better MSE with a score of 7.103 million

Score that Simple Linear Regression

[189]
```
# get the MSE score of Linear Regression
predicted = x_model.predict(X)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
# # Gets around 7.107 mil
```

```
We need to get a MSE better than 7107309.7162
```

[190]
```
# get the MSE score of Linear Regression with p bigger than 0.3 removed
predicted = x_model_Pbigger3.predict(X_new_Pbigger3)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7395836.6957
```

[191]
```
# get the MSE score of Linear Regression with p bigger than 0.5 removed
predicted = x_model_Pbigger5.predict(X_new_Pbigger5)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7255746.4066
```

Ridge cv

- We see that Ridge CV with P bigger than 0.5 gets a better MSE
- This time, the MSE of all the models increased from the Simple Linear Regression which would make Simple Linear Regression so far.

Score that Ridge CV

[192]
```
# get the MSE score of RGD CV
x_model2 = linear_model.RidgeCV(alphas=[0.8, 1.0, 10.0])
x_model2.fit(X, y)
coeffs_XRCVr = np.array(list(x_model2.intercept_.flatten()) + list(x_model2.coef_.flatten()))
coeffs_XRCVr = list(coeffs_XRCVr)

predicted = x_model2.predict(X)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7394603.2487
```

```
# get the MSE score of RGD CV with p bigger than 0.3 removed
x_model2 = linear_model.RidgeCV(alphas=[0.8, 1.0, 10.0])
x_model2.fit(X_new_Pbigger3, y)
coeffs_XRCVr = np.array(list(x_model2.intercept_.flatten()) + list(x_model2.coef_.flatten()))
coeffs_XRCVr = list(coeffs_XRCVr)

predicted = x_model2.predict(X_new_Pbigger3)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7540689.2918
```

[194]
```
# get the MSE score of RGD CV with p bigger than 0.5 removed
x_model2 = linear_model.RidgeCV(alphas=[0.8, 1.0, 10.0])
x_model2.fit(X_new_Pbigger5, y)
coeffs_XRCVr = np.array(list(x_model2.intercept_.flatten()) + list(x_model2.coef_.flatten()))
coeffs_XRCVr = list(coeffs_XRCVr)

predicted = x_model2.predict(X_new_Pbigger5)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7390408.9375
```

Elastic Net

- We see that Elastic Net with all the features gets a better MSE
- Again, the MSE score increased again to have the smallest MSE score now be around 8.184 million

Score that Elastic Net

[195]
```
# get the MSE score of Elastic Net Regression
x_model3 = linear_model.ElasticNet(alpha=3.0, l1_ratio=0.9)
x_model3.fit(X, y)
coeffs_XENr = np.array(list(x_model3.intercept_.flatten()) + list(x_model3.coef_.flatten()))
coeffs_XENr = list(coeffs_XENr)

predicted = x_model3.predict(X)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```
```
We need to get a MSE better than 8183836.258
```

[196]
```
# get the MSE score of Elastic Net Regression with p bigger than 0.3 removed
x_model3 = linear_model.ElasticNet(alpha=3.0, l1_ratio=0.9)
x_model3.fit(X_new_Pbigger3, y)
coeffs_XENr = np.array(list(x_model3.intercept_.flatten()) + list(x_model3.coef_.flatten()))
coeffs_XENr = list(coeffs_XENr)

predicted = x_model3.predict(X_new_Pbigger3)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```
```
We need to get a MSE better than 9527632.3809
```

```
# get the MSE score of Elastic Net Regression with p bigger than 0.5 removed
x_model3 = linear_model.ElasticNet(alpha=3.0, l1_ratio=0.9)
x_model3.fit(X_new_Pbigger5, y)
coeffs_XENr = np.array(list(x_model3.intercept_.flatten()) + list(x_model3.coef_.flatten()))
coeffs_XENr = list(coeffs_XENr)

predicted = x_model3.predict(X_new_Pbigger5)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```
```
We need to get a MSE better than 9320441.7816
```

ARD Regression

- We see that ARD Regression with all the features gets a better MSE
- It seems to get similar results to the Ridge CV but still worse results compared to a simple linear regression

Score that ARD Regression

```
[ ]  1 # get the MSE score of ARD Regression
     2 x_model4 = linear_model.ARDRegression()
     3 x_model4.fit(X, y)
     4 coeffs_XARDr = np.array(list(x_model4.intercept_.flatten()) + list(x_model4.coef_.flatten()))
     5 coeffs_XARDr = list(coeffs_XARDr)
     6
     7 predicted = x_model4.predict(X)
     8 error = (predicted - y)**2
     9 MSE_SLR = round(error.mean(), 4)
    10 print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7423041.9718
```

```
[199]  1 # get the MSE score of ARD Regression with p bigger than 0.3 removed
       2 x_model4 = linear_model.ARDRegression()
       3 x_model4.fit(X_new_Pbigger3, y)
       4 coeffs_XARDr = np.array(list(x_model4.intercept_.flatten()) + list(x_model4.coef_.flatten()))
       5 coeffs_XARDr = list(coeffs_XARDr)
       6
       7 predicted = x_model4.predict(X_new_Pbigger3)
       8 error = (predicted - y)**2
       9 MSE_SLR = round(error.mean(), 4)
      10 print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7573336.6613
```

```
[200]  1 # get the MSE score of ARD Regression with p bigger than 0.5 removed
       2 x_model4 = linear_model.ARDRegression()
       3 x_model4.fit(X_new_Pbigger5, y)
       4 coeffs_XARDr = np.array(list(x_model4.intercept_.flatten()) + list(x_model4.coef_.flatten()))
       5 coeffs_XARDr = list(coeffs_XARDr)
       6
       7 predicted = x_model4.predict(X_new_Pbigger5)
       8 error = (predicted - y)**2
       9 MSE_SLR = round(error.mean(), 4)
      10 print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7573332.0574
```

SGD Regression

- We see that SGD Regression with all the features gets a better MSE
- While the other explode and get a massive MSE which is not good

Score that SGR Regression

[201]
```
# get the MSE score of SGD Regression
x_model5 = linear_model.SGDRegressor()
x_model5.fit(X, y)
coeffs_XSGDr = np.array(list(x_model5.intercept_.flatten()) + list(x_model5.coef_.flatten()))
coeffs_XSGDr = list(coeffs_XSGDr)

predicted = x_model5.predict(X)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 7360460.9773
```

[202]
```
# get the MSE score of SGD Regression with p bigger than 0.3 removed
x_model5 = linear_model.SGDRegressor()
x_model5.fit(X_new_Pbigger3, y)
coeffs_XSGDr = np.array(list(x_model5.intercept_.flatten()) + list(x_model5.coef_.flatten()))
coeffs_XSGDr = list(coeffs_XSGDr)

predicted = x_model5.predict(X_new_Pbigger3)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 3.0455652131742696e+32
```

```
# get the MSE score of SGD Regression with p bigger than 0.5 removed
x_model5 = linear_model.SGDRegressor()
x_model5.fit(X_new_Pbigger5, y)
coeffs_XSGDr = np.array(list(x_model5.intercept_.flatten()) + list(x_model5.coef_.flatten()))
coeffs_XSGDr = list(coeffs_XSGDr)

predicted = x_model5.predict(X_new_Pbigger5)
error = (predicted - y)**2
MSE_SLR = round(error.mean(), 4)
print(f"We need to get a MSE better than {MSE_SLR}")
```

```
We need to get a MSE better than 9.054380471406017e+32
```

Linear Regression Again

The one with all the features gets MSE of 7.107 million

The one with the P bigger than 0.3 removed gets MSE of 7.396million

The one with the P bigger than 0.5 removed gets MSE of 7.256million

```
[ ]
2 predicted = x_model.predict(X)
3 error = (predicted - y)**2
4 MSE_SLR = round(error.mean(), 4)
5 print(MSE_SLR)
6 print(f"\n\nWe need to get a MSE better than {MSE_SLR}")
7 # # Gets around 7.107 mil
```

```
7107309.7162


We need to get a MSE better than 7107309.7162
```

```
1 # get the MSE score of Linear Regression with p bigger than 0.3 removed
2 predicted = x_model_Pbigger3.predict(X_new_Pbigger3)
3 error = (predicted - y)**2
4 MSE_SLR = round(error.mean(), 4)
5 print(MSE_SLR)
6 print(f"\n\nWe need to get a MSE better than {MSE_SLR}")
```

```
7395836.6957


We need to get a MSE better than 7395836.6957
```

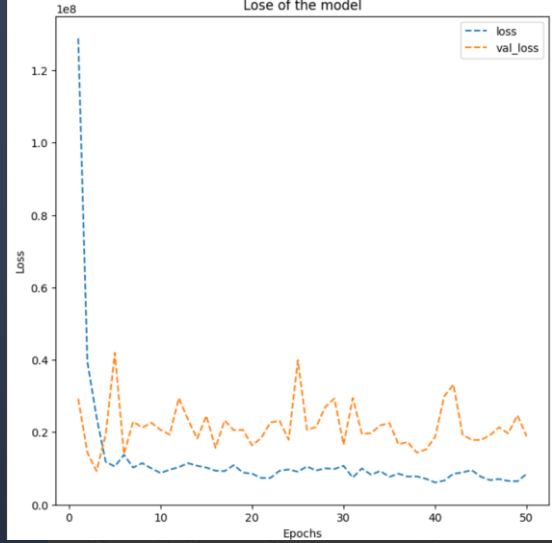
```
[ ]
1 # get the MSE score of Linear Regression with p bigger than 0.5 removed
2 predicted = x_model_Pbigger5.predict(X_new_Pbigger5)
3 error = (predicted - y)**2
4 MSE_SLR = round(error.mean(), 4)
5 print(MSE_SLR)
6 print(f"\n\nWe need to get a MSE better than {MSE_SLR}")
```

```
7255746.4066
```

Our model results

We would sometimes get a one of them to be smaller than the linear regression model (in this case X_test) and one or two that are really bad (in this case just X_val).

We typically get one or two that is close but most of the time above the linear regression model (like with the X_train)



Score that we got from our model

```
[49]  1 prediction = model_1.predict(X_train)
      2 error = (prediction.T[0] - y_train)**2
      3 MeanSquaredError = round(error.mean(), 4)
      4 print(f'Mean Squared Error is {MeanSquaredError}')

      6/6 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step
      Mean Squared Error is 7763305.7227
```

```
[50]  1 prediction = model_1.predict(X_test)
      2 error = (prediction.T[0] - y_test)**2
      3 MeanSquaredError = round(error.mean(), 4)
      4 print(f'Mean Squared Error is {MeanSquaredError}')

      1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 247ms/step
      Mean Squared Error is 6458757.7719
```

```
[51]  1 prediction = model_1.predict(X_val)
      2 error = (prediction.T[0] - y_val)**2
      3 MeanSquaredError = round(error.mean(), 4)
      4 print(f'Mean Squared Error is {MeanSquaredError}')

      1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 298ms/step
      Mean Squared Error is 18855259.6316
```

Our model

- Not as good as linear regression
- Results never seems to be better then linear regression and seem to overfit with no apparent way to really make it reliable


* see ya.